ENS 491-492 – Graduation Project

Final Report

Project Title:

An LLM-based Course Assistant for Sabancı University

Group Members:

Burak Yılmaz

Efe Ballar

Göktuğ Aygün

Timur Can Turut

Supervisors:

Hüsnü Yenigün

İnanç Arın

Kamer Kaya

Date: 18.05.2025

1 Executive Summary

The aim of our project was to develop a RAG model that would be able to answer students' questions based on the documents provided by the instructors. Students will be able to enter a specific URL within the university network and interact with the Retrieval-Augmented Generation (RAG) by sending their questions. A VLLM model will be responsible for handling the questions and answering them based on the chunks it has in its vector database. We had hoped to deliver an alternative solution for both instructors and students who have questions regarding the course material, allowing students to have a flexible schedule for asking questions and decreasing the amount of work instructors have to do to answer them. A VLLM model was installed on the remote server. Students will interact with the React frontend and will be able to select their courses. Instructors will have a different screen where they can upload or remove documents from the vector database, keeping it up to date. The backend will then use the chunks generated by FAISS, send the most relevant chunks to the model, and obtain an answer. The backend is also responsible for logging the conversation metadata in a remote MongoDB database. MongoDB will keep track of user IDs and will store all conversations of a specific user with the LLM. The product will be accessible only via a login system. We hope to deliver a functional system that will help students gain a better grasp of their courses. The current implementation meets the primary goals we set out to achieve, and we believe it can serve as a dependable support tool for both students and instructors. Our focus now is on ensuring stability and ease of use before deployment.

2 Problem Statement

The original problem addressed by this project was the dependency of students—who wish to ask questions related to course content—on instructors. Currently, students are limited by instructors' availability or must rely on external resources that may not align with the specific materials used in the course. To solve this, we aimed to build an automated question-answering system that could respond to students' queries based directly on course documents provided by instructors. Instead of fine-tuning a large language model—which can be computationally expensive and may yield inconsistent results due to reliance on general training data—we opted to implement a RAG architecture. This approach ensures that the generated answers are grounded in instructor-approved documents, improving both relevance and reliability. The system was designed to include a functional React-based frontend and a backend pipeline capable of retrieving relevant document chunks using FAISS, querying a VLLM model, and returning responses based on the chunks that were sent to the model.

The entire platform would be deployed on a remote server within the university network, allowing students to access it freely and submit their questions at any time. Previous iterations of this idea were attempted by earlier students; however, those efforts remained at the prototype stage and were never deployed for real-world use. Our primary goal was to go beyond experimentation and deliver a fully functional, deployable system that can be maintained and expanded upon in the future.

2.1 Objectives/Tasks

2.1.1 Build a RAG-based backend system

The primary objective was to design two systems, a backend system that integrates a RAG pipeline and a React frontend that would form the UI part of the product. The backend system retrieves the most relevant chunks from course documents using FAISS, sends these to the VLLM model, and returns a generated answer. This required setting up document chunking, vectorization, and similarity-based retrieval logic. Frontend is responsible for using the endpoints to get the answers from the backend and display it to the users and provide an interface that can be used for instructors so that they can control the course materials they provide.

2.1.2 Control the scope of the answers

One of our main objectives was to ensure the model responds to the question if and only if a relevant chunk exists in its vector database. We had to use *content prompt* and additional functional calls to ensure the model never checks its training data and only uses the relevant chunks it received from the backend to answer the questions. This was also done to ensure a non-relevant answer is not produced by the model.

2.1.3 Develop a React-based frontend

A user-friendly frontend was developed using React. This interface allows students to log in, select their enrolled courses, and submit questions to the system. The goal was to ensure accessibility and a clean UI/UX to encourage frequent use. This is important especially considering the fact that not all of our students and instructors are familiar with these kinds of systems. We wanted to provide an UI as user-friendly as possible to encourage them to use the product.

2.1.4 Create an instructor dashboard

Instructors have a separate dashboard where they can upload or remove course documents. This feature enables them to keep the vector database up to date, ensuring the model's responses are based on the most current material. This will also ensure that the scope of the answers are limited, and the model will not use its training data to answer the questions. The vector database will be checked and since it consists of documents uploaded only by the instructors, the scope will be limited by what they upload, preventing excessive or irrelevant answers.

2.1.5 Implement secure login and authentication system

To control access and ensure data privacy, a secure login system was implemented. Role-based authentication distinguishes between student and instructor users, limiting features according to their roles. This will also ensure that a user can only access the system via their Sabancı credentials and this will prevent any type of malicious intent. Even if an attack were to occur, it can be tracked by the userID who performed that attack, disencouraging any possible malicious moves against the system.

2.1.6 Integrate database for logging conversations

A MongoDB database was integrated to log metadata about each conversation. This includes user ID, timestamps, course selection, and query history, providing a backend record of interactions for potential future analysis or debugging. This will be used to provide a more consistent experience for each user.

2.1.7 Deploy VLLM model on a remote server

The VLLM model was deployed on a high-performance remote server with GPU capabilities. This setup ensures low latency and high availability for generating responses. As the VLLM, as a library, supports concurrency, more than one individual will be able to interact with the model without any race conditions or any crashes.

2.2 Realistic Constraints

2.2.1 Computational Resource Constraints

Large language models like VLLM require significant memory and GPU capacity. We addressed this by deploying the model on a remote server provided by the university that met the necessary hardware specifications. We also optimized the model's context window and

batch sizes to balance performance with resource usage. This ensures the server will work independent of the developers and students will be able to interact with the model regardless of developer availability.

2.2.2 Network and Access Limitations

Due to privacy concerns and resource management, the system was made accessible only within the university network. This ensures secure usage and prevents overload from external or unauthorized access. The students within the campus can freely connect to the model while the students outside the campus can use SuVPN to access the model, allowing all students to use the model as long as they are connected to the network.

2.2.3 Security and Privacy Concerns

The login system was implemented to keep track of who is actively using the system. A user can't access the model without providing their credentials. This will allow maintainers to keep track of the activities performed by the active users allowing additional surveillance. Handling student queries comes with data privacy obligations. We implemented role-based login (instructor, student), secure API endpoints to ensure that data is kept private and tamper-proof. Since the model only tells which pages are relevant rather than providing the documents itself, the model hasn't got any issues regarding document privacy.

2.2.4 Usability Constraints

Some instructors may not be familiar with modern web systems. To address this, the frontend was built with a focus on simplicity and intuitive design. We minimized technical jargon and ensured clarity in button labeling and navigation.

2.2.5 Economic Constraints

We prioritized open-source tools to keep costs low. FAISS, MongoDB, React, and VLLM were chosen over commercial alternatives to remain within the project's limited budget, without compromising on functionality.

2.2.6 Scalability Constraints

The system needed to scale to multiple users and courses simultaneously. We designed modular backend services and used efficient FAISS indexing to maintain performance even as the number of stored documents and active users increases. FAISS will only reconstruct

the vector database when a document is deleted or uploaded by an instructor. The process of creating the vector database will not be called so frequently. This will ensure that the system will not be performing redundant tasks and will only focus on creating answers.

3 Methodology

The methodology for the project follows agile project development strategy. The developer team has participated in multiple meetings with the project owner, supervisors for Sabancı University, analysing the project. Through weekly meetings over two semesters, the project development is traced and evaluated. In short, the project comprises following major stages: requirements analysis, system design, prototype development, experiment design, and testing & validation.

3.1 Requirement Analysis

Based on the requirements obtained from the supervisors and past prototypes, the team created a list of requirements that is subject to change throughout the year. Thus, a methodology capable of maintaining changing requirements is selected. Weekly meetings made it possible to trace the development besides receiving updates for both parties.

3.1.1 Conceptual Design

The team adopted a Retrieval-Augmented Generation (RAG) architecture, combining a pre trained LLM (initially Meta's LLaMA, later evolving to Qwen-based models) with a vector database (initially ChromaDB / later evolving to Meta's FAISS) and a NoSQL document store (MongoDB). Key design decisions included:

- Leveraging vector embeddings for accurate similarity search. Eventually, this is obtained via SentenceTransformer
- Incorporating quantization methods (e.g., GPTQ-Int4) to reduce model footprint while preserving performance.
- Defining secure, role-based access controls in the front-end and back-end structures (students, instructors, admins).

3.2 System Design

3.2.1 Conceptual Design

Based on the requirements specified in the Requirement Analysis section and real life limitations, the team developed different prototypes and experimented with different tools and technologies. Main development that shaped the structure of the project are as follows:

- Flask based Python back-end structure, in addition to React.js based front-end web application
- NoSQL based data storage that supported necessary operations
- Vector-embedding structure that made it possible for RAG Application to work efficiently

3.2.2 Preliminary & Detailed Design

- A Flask-based REST API backend was paired with a React.js frontend supporting document uploads (PDF, DOCX) and query submission
- Text chunking strategies were refined to maintain context across document sections
- Threshold and similarity parameters for retrieval were identified as crucial tuning knobs for balancing precision and recall. The team experimented with different threshold levels and different similarity techniques (such as cosine similarity), and fine-tuned these parameters

3.2.3 Logical Architecture & UML Diagrams

The system is built around a set of core entities that represent real-world concepts within the application domain. These include *User*, *Admin*, *Course*, *Chat*, *Message*, and *CourseFile*. Each entity encapsulates relevant attributes and methods, forming the foundation for data modeling, logic, and relationships across the system. Below you can see the entity-relationship diagram of the project.

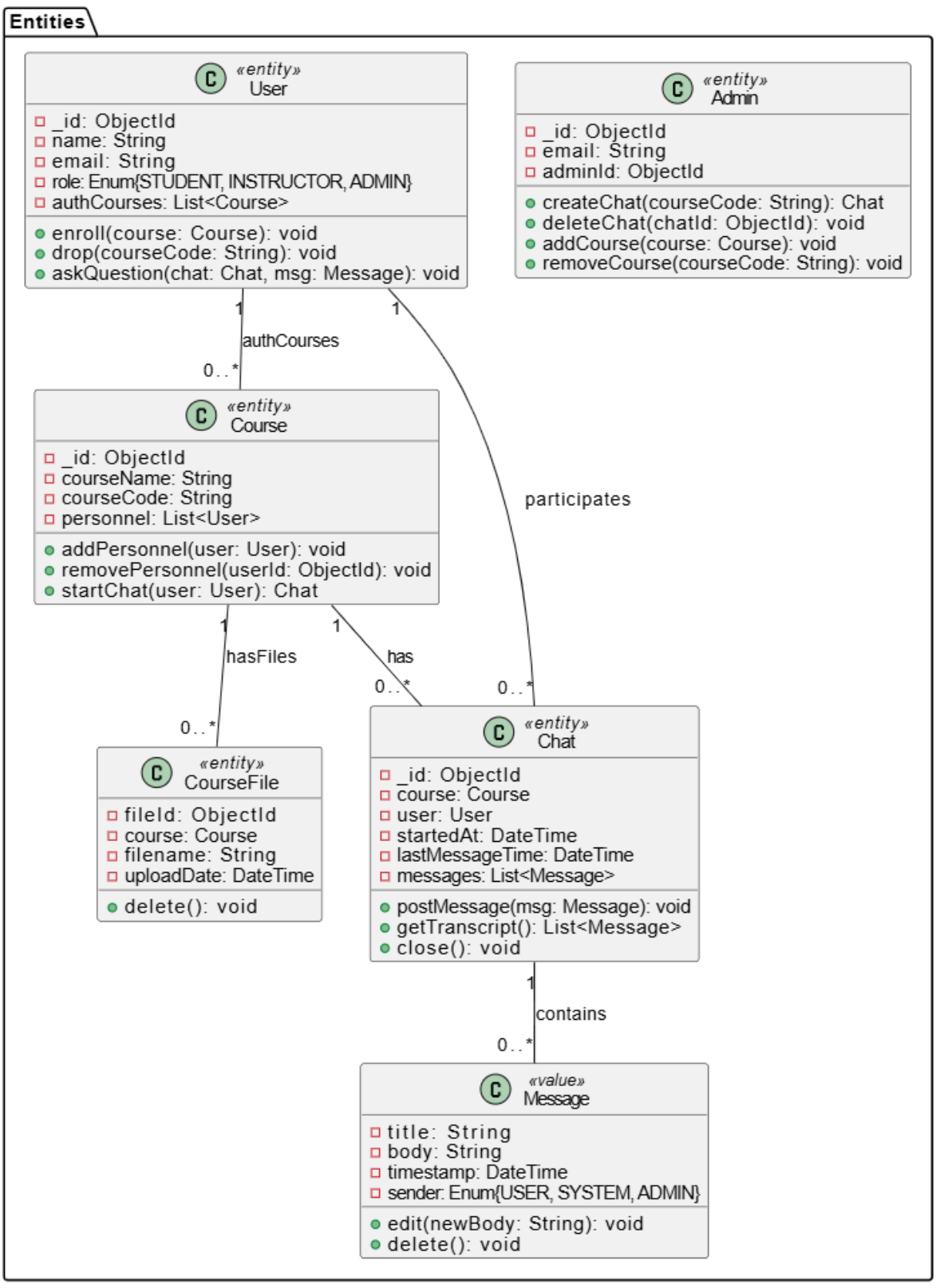

Figure 1: Entity-Relationship Diagram

Controllers serve as intermediaries between the user interface and the application logic. Each controller corresponds to a group of RESTful API endpoints and is responsible for managing requests, invoking appropriate entity operations, and returning structured responses. Below lies the controllers; *AuthenticationController*,*AdminController*, *UserController*, *CourseController*, *FileController*, and *QueryController*, each aligned with specific functional modules in the system.

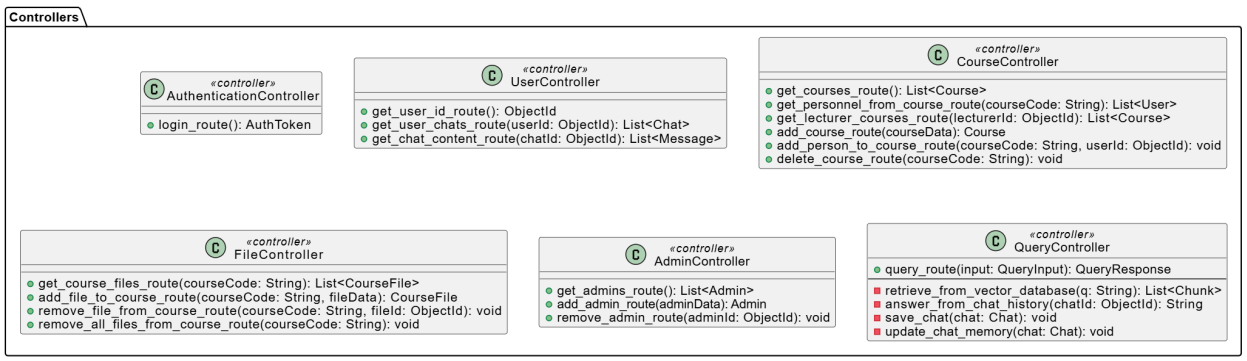

Figure 2: Class Diagram of Controllers

3.3 Prototype Development

3.3.1 Integration of Components

The team built end-to-end communication between frontend and backend, enabling:

- Operation relating course materials by authorized instructors and assistants
- Vectorization and storage of these material contents in the vector database, FAISS
- Student queries routed through the RAG pipeline to generate answers with source citations

3.3.2 Quantization Implementation

A primitive quantization pipeline was developed to evaluate trade-offs between model size, latency, and accuracy, preparing the system for deployment on constrained hardware. Another decision that is made by the project team is to utilize a central server that will deploy the LLM model, eventually Deepseek model with 32B parameters. Other projects that also utilize LLM models and RAG system design are to send requests using VLLM structure, enabling concurrency as well.

3.4 Experiment Design

Two controlled studies were planned to optimize system performance:

3.4.1 Threshold Parameter Optimization

- **Inputs:** 100 representative student queries with known 'gold' answers
- **Variable:** Similarity threshold for retrieval
- **Outputs:** Precision, recall, and F1 score at each threshold

3.4.2 Quantization Impact Analysis

- **Inputs:** Baseline and quantized models processing identical query sets
- **Variable:** Model bit-width, hardware configuration (server vs. desktop)
- **Outputs:** Response latency and answer accuracy

Experiments of quantization are conducted and outcomes are evaluated. However, system tests are not conducted because of the delays that occurred as a result of unforeseen obstacles and problems that slowed down the process.

3.5 Testing & Validation

3.5.1 White-box Testing

Developers with full visibility into the prompt mechanisms were planned to conduct systematic debugging to identify inconsistencies and improve prompt templates. They documented recurring failure patterns and iteratively adjusted the prompts to enhance model behavior. This process also facilitated the creation of reusable prompt components for future deployments.

3.5.2 Black-box Testing

External testers (students and TAs) evaluated usability and response relevance, providing feedback on UI/UX and answer quality. Their insights highlighted areas where the interface could be streamlined and where the model struggled with domain-specific queries. This feedback loop was integral to refining both functionality and clarity.

3.5.3 Live-Server Verification

Post-tuning, a live server setup was used to validate improvements under real-world load conditions. Performance metrics such as latency, accuracy, and throughput were continuously monitored. This testing phase ensured the system remained stable and responsive under varying usage patterns.

4 Results & Discussion

4.1 Realization of Objectives

4.1.1 Develop the AI Model (Objective 1)

Initial objective was to experiment with small models via Hugging Face to combine and chain these models to obtain a working model. However, in later stages the team decided to utilize open-source models by Meta. A working RAG pipeline was implemented using Qwen2.5-32B-Instruct-GPTQ-Int4 (later DeepSeek-R1-Distill-Qwen-14B) integrated with FAISS and MongoDB. Embedding experiments confirmed reliable retrieval of course materials.

4.1.2 User Interface & Backend Integration (Objectives 2 & 5)

- A React-Flask (and earlier Node.js) frontend now supports secure file uploads, role-based dashboards, and chat functionality.
- Endpoints for query submission, authentication (JWT/Google Auth), and document management are operations. These operations include endpoints for all kinds of users, students, TAs, instructors and admins.

4.1.3 Implement Query Functionality (Objective 3)

- We built a React-based chat widget connected to our Flask REST API. Students can now type free-form questions, which are routed through the RAG pipeline to the LLM and vector store.
- Model outputs are evaluated and model prompts are shaped in order to improve answer accuracy.
- Queries trigger on-the-fly retrieval of the top k most similar text chunks, which the LLM uses as context—ensuring answers remain grounded in instructor-provided materials.

4.1.4 Source Attribution (Objective 4)

The assistant returns answers annotated with instructor-approved source snippets, fulfilling traceability requirements.

4.1.5 Privacy & Deployment (Objectives 7 & 8)

- Environment-based configs and CORS policies ensure secure handling of user data.
- Although full deployment to Sabancı servers remains pending, the system is containerized for future rollout.

4.2 Deviations from Initial Objectives

- The core LLM choice evolved from small models from Hugging Face to Meta's LLaMA eventually to Qwen-based Distilled Deepseek Model, reflecting hardware considerations.
- Embedding backends shifted from ChromaDB to FAISS for scalability under multiple concurrent users.
- Due to a potential shift from the Central Authentication System (CAS) to an alternative system, the team decided to use Google Authentication. However, the final decision has not yet been made, making it impossible to finalize the authentication system. This delay also affects other aspects and modules of the project.

4.3 Project Completion Status

- Core architectural components, quantization pipeline, and basic UI/UX are in place.
- Stress testing under high load, UI refinements (additional features and front-end design), advanced permission controls, end-to-end deployment, and final documentation are scheduled through May 2025.
- The milestones for system optimization, security enhancements, and user testing align with the Phase 6 deliverables.

4.4 Contribution to State-of-the-Art

- Unlike generic chatbots, this assistant tightly integrates quantized LLMs with curriculum-specific retrieval and verified source attribution.

- The combination of low-bit quantized models, configurable text-chunking, and modular microservices architecture offers a blueprint for scalable, cost-effective academic AI tools.
- By offloading repetitive queries from human TAs and providing 24/7 support, the system enhances educational efficiency while maintaining academic rigor.

Artificial Intelligence technologies are progressively getting more popular and this integration into casual life has shown an exponential growth. Utilization of these technologies for such problems demonstrate the true power of the field and play a pioneering role for future projects and developments. Overall, the project has demonstrated its feasibility through a functional prototype, addressed initial design challenges via iterative experimentation, and laid the groundwork for a robust, scalable course assistant that meaningfully advances AI-driven academic support. As the problems specified in this report are resolved, the project will be further developed satisfying the requirements and achieving the desired end-result.

5 Impact

Without any restrictions on freedom of use, the LLM-based Course Assistant has an influence on three interconnected fronts: scientific, technical, and socioeconomic. It also has obvious inventive and commercial possibilities.

5.1 Scientific Impact

The project advances pedagogical AI research by demonstrating how a Retrieval-Augmented Generation (RAG) pipeline can be tightly integrated with multi-format course materials while preserving high contextual relevance. In particular, the adoption of FAISS for similarity search together with SentenceTransformer embeddings illustrates an effective solution to the challenge of balancing inference efficiency and retrieval accuracy in academic settings. Moreover, the configurable text-chunking strategy ensures that long and heterogeneous documents (PDF, Word, PPTX, text) are processed without loss of semantic coherence, contributing to ongoing discussions on scalable context-management techniques in large-model applications.

5.2 Technological Impact

The system's microservices architecture—Flask REST API backend, React frontend, and MongoDB document store—provides a production grade blueprint for deploying secure, scal-

able LLM based services. Key technological contributions include:

- Use of FAISS for sub second similarity lookups, paired with all MiniLM L6 v2 embeddings, showcases how to achieve low latency retrieval at scale.
- Implementation of authentication, role based access control, environment variable configuration, and CORS policies establishes a robust security posture suitable for managing sensitive educational data.
- Administrative ability to swap base LLMs (e.g., DeepSeek-R1-Distill-Qwen-14B) and the modular separation of concerns enable straightforward horizontal scaling and integration with external systems.

5.3 Socio-Economic Impact

The assistant immediately addresses educational assistance limitations by serving as a "supplementary TA" anytime, anywhere:

- Improved Access: Students who are unable to attend regular office hours receive prompt, on-demand explanations, which lessens disparities.
- Operational Efficiency: By automating repetitive questions, human teaching assistants may concentrate on more difficult educational responsibilities, which may reduce staff expenses or allow for resource redistribution.

5.4 Internal Operational and Value-Add Aspects

With deployment limited to Sabancı University, the project's "entrepreneurial" objective switches from commercialization to institutional value maximization:

- Because the system is made to integrate seamlessly with Sabancı University's current learning management system, there is less overhead for support and IT personnel. Phased rollout by department, term, or course is made possible by its modular microservices design.
- By empowering the teaching staff and IT team at Sabancı University to maintain and expand the platform, training seminars and documentation promote internal competence in AI-driven instructional tools.
- The university may go on developing the software on its own because all of its components are open source and protected under permissive licenses.

5.5 Freedom to Use Considerations

Only open source, permissively licensed dependencies are found when all software components (FAISS, MongoDB, Flask, and React) and data sources are examined. Academic deployment is made possible by the project's lack of restricted models or private datasets, as well as its absence of Freedom to Use limitations.

6 Ethical Issues

Although no components of the LLM-based Course Assistant rely on patented or toxic materials, several ethical considerations must be acknowledged and managed:

6.1 Data Privacy & Confidentiality

The system processes sensitive student data (queries, course enrolment, chat histories). A breach could expose private academic records. To mitigate this, we enforce authentication, role based access control, environment variable configuration, and CORS policies, and we have explicitly targeted "Ensure data privacy and compliance with regulations." as a project objective.

6.2 Potential for Hallucinations & Misinformation

The assistant faces the potential of providing inaccurate or fraudulent explanations, as is the case with all generative LLMs. Inaccuracies can spread if students depend on such outcomes without question. To increase accuracy, we use similarity thresholding, do thorough testing, and reveal source attributions for transparency.

6.3 Academic Integrity

Offering students on-demand responses increases the likelihood that they would exploit the aid to avoid actual learning. In order to address this, we will:

- Promote acknowledging the helper as an additional resource, not a substitute for individual work
- Create usage guidelines in accordance with the honor code of Sabancı University

6.4 Bias and Fairness

Social biases in training data can be reflected in pre-trained algorithms. Student groups may be offended by insensitive answers. Before going live, we'll check outputs for bias, retrain or filter problematic information, and get input from a variety of pilot users.

7 Project Management

7.1 Methodology and Tools

- **Version Control:** We used Github to track different version of codes throughout the development process, besides utilizing branch management and other properties. This enabled team members to maintain the source code more efficiently and securely. Through clear communication and coordination, the code is stored at one place enforcing common practices. Some leaks have occured exposing some sensitive information, however these problems are solved by some practical solutions by the team members. One example can be given as .env file being included in a push operation as it was not included in .gitignore file. This is solved by changing sensitive information such as passwords. Below are the links for backend and frontend repositories used in the project.

- **F**rontend: https://github.com/timurturut/sugpt

- **B**ackend: https://github.com/EfeBallar/LLM-Based-Course-Assistant

7.2 Initial Plan vs. Reality

- **Kick-off (September 2024).** We sketched a classic waterfall Gantt chart: data collection in October, backend prototype by December, full system by April.

- **Pivot to agile (October 2024).** After the first code spike we saw requirements changing every week, so we switched to two-week sprints with a Trello board and short stand-ups.

- **Mid-term reset (February 2025).** Hardware limits forced us to drop an early model and move to a quantized one. We re-planned the spring task to keep the core RAG pipeline.

7.3 Roles and Communication

- Each member "owned" one slice (frontend, backend, DevOps, etc.) but most of the time there were more than one person in the same area.
- Weekly meetings with supervisors kept us honest and gave fast feedback.
- We logged every decision in a shared notepad so no detail lived only in someone's head.

7.4 Lessons Learned

- **Continuous improvement is the way to achieve a great result.** It is not possible to satisfy all the requirements with a single shot. Mistakes are supposed to be made, prototypes are supposed to be developed and changed to achieve the best results in the best way possible. Starting with a smaller but reliable product and improving it over time is much more reasonable to finish the project at once. In fact, latter one is almost impossible.
- **Plan for failure, not hopes.** Always have backup of your work. You will make lots of mistakes and the last thing you would want to do is to find the version your code was working.

8 Conclusion and Future Work

8.1 Achievements

- A working Retrieval-Augmented Generation assistant that answers course questions using instructor documents, with source citations.
- A React dashboard for students and a separate panel for instructors to upload or remove material.
- A quantized 14 B-parameter model running on a university GPU server

8.2 Current Limitations

- Authentication is still Google-based; integration with the university's CAS awaits a final decision.
- Stress testing is limited to small user groups, so peak-hour performance is unverified.
- UI/UX is functional but plain—mobile layout and accessibility checks are pending.

8.3 Next Steps

1. Containerize the stack, and roll out to one pilot course.
2. Connect the assistant to the university's learning-management system so students do not need a second login.
3. Conversation summaries could be emailed to students.
4. Developing an instructor analytics showing the most-asked questions.

8.4 Long-Term Vision

Over the next three to five years, we imagine the assistant growing far beyond a single course or even a single faculty. In its mature form, it could act as an information desk for the entire campus community. A first year student could ask, "When is the add–drop deadline?" and get the registrar's official answer with a link to the policy page, then immediately follow up with "What electives fit my minor?" and receive a filtered list from the advising office. A graduating senior might switch context and ask for resume tips, internship leads pulled from the career center's database, or a quick comparison of master's programs abroad. Faculty and staff could query procurement rules, HR forms, or research-grant guidelines without hunting through outdated PDFs. Because every response would cite its source, handbook pages, or verified administrative memos, the assistant would double as a gentle nudge toward better document hygiene: If an answer is wrong, the underlying document is probably stale and needs fixing. Metrics on unanswered or low-confidence queries would highlight knowledge gaps and drive continuous content updates. Equally important, the project would serve as a live testbed for responsible AI in education. Future teams could experiment with bias audits, differential privacy, or on-device inference, publishing their findings so other universities learn what works and what to avoid, when rolling out conversational AI at scale. In short, the assistant could evolve from a helpful chatbot into both campus collective memory and a showcase of ethical and transparent AI practice.

9 How to use LLM-Based Course Assistant

Below are the steps to follow in order to start the product. This way all the students with the domain @sabanciuniv.edu will be able to use it via *dolap.sabanciuniv.edu*. Note that the Large Language Model should be running in its dedicated port. This can be ensured by the system administrator and checked by using the following command:

nvidia-smi
Any running process can be seen as a result of this command.

9.1 Steps:

1. Establish an ssh connection. Sensitive information such as password or server address will not be disclosed here due to security concerns.

2. Navigate to cont folder with *cd cont/* command.

3. If it is the first time running the app, you should use the command: *docker compose up –build*
   This is also valid if there are some changes made in docker-compose.yml or dockerfiles. It is not the case for changes made in other files such as .py files. Just a simple *docker compose up* is necessary.

4. Both the backend server and frontend product should be working by now. Students can connect using the link: http://dolap.sabanciuniv.edu:3101

   There are different endpoints such as */login*, */admin* or just / for queries. Users should be able to navigate through these endpoints using the buttons provided in the interface. The developers can review the endpoints in app.py file.
   One important aspect here is that the link provides http connection, **not https**. Another one is the use of top level domain being *dolap.sabanciuniv.edu*. Since Google Authentication does not accept an IP-address for authorization, this choice had to be made.

10 Images From The Application

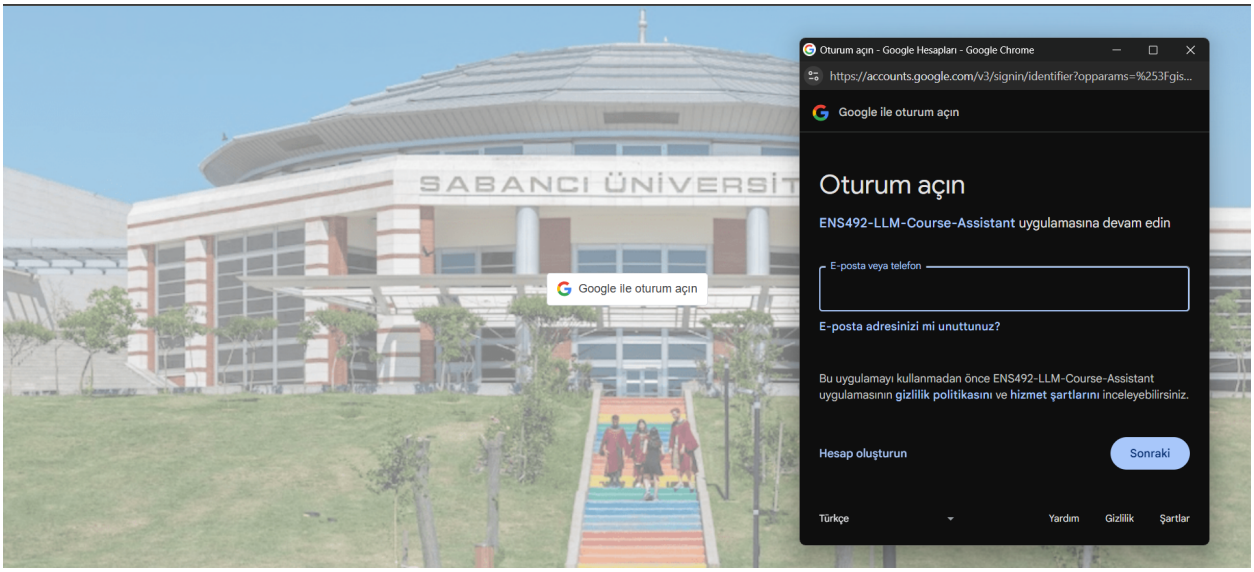

Figure 3: Login Page for Using Google Authentication

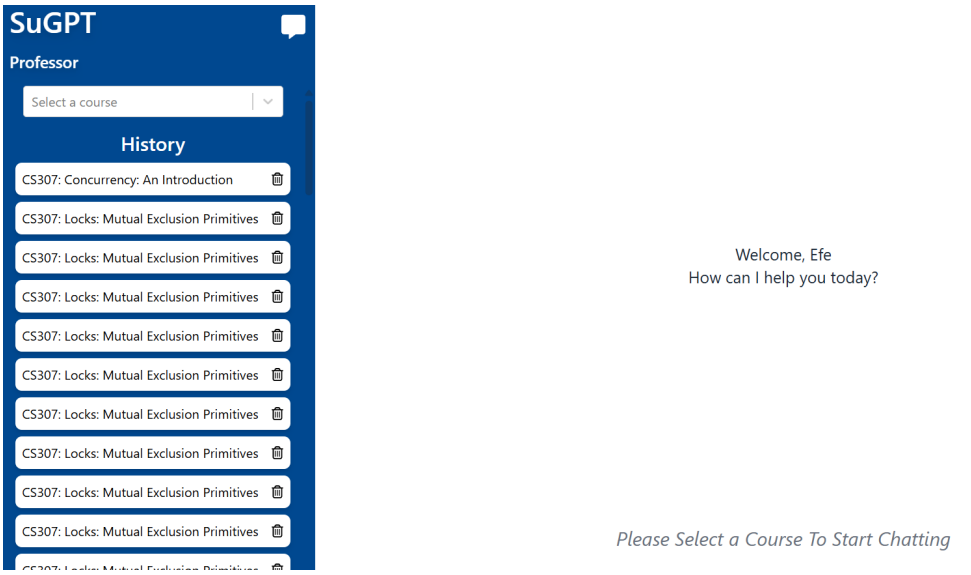

Figure 4: Main Screen Without A Course Selection

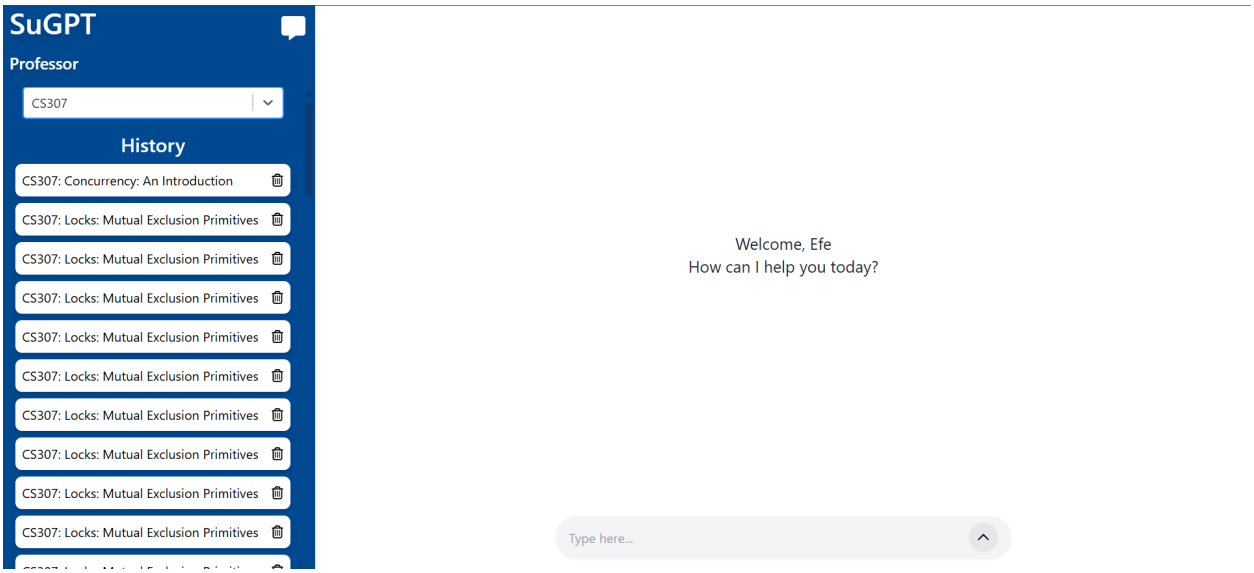

Figure 5: Main Screen With A Course Selection

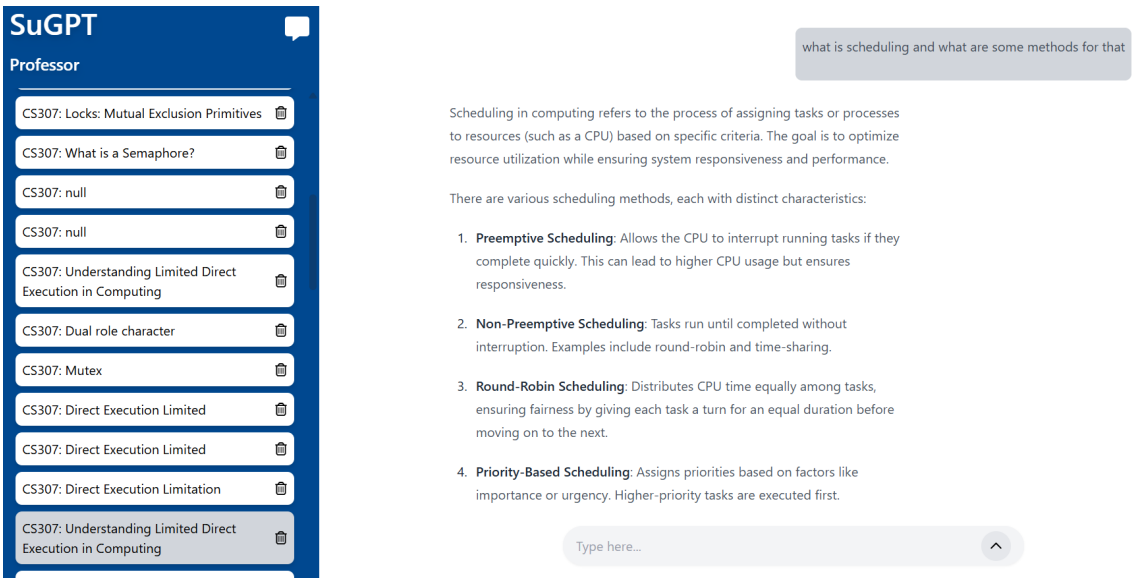

Figure 6: Main Screen Displaying A Response

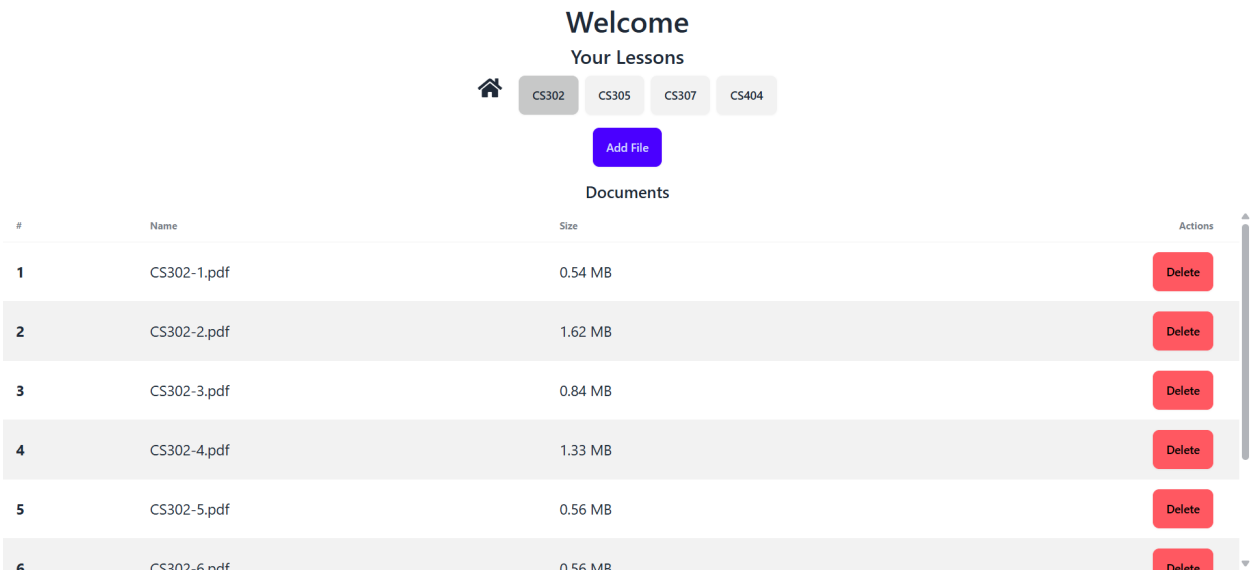

Figure 7: Professor Screen To Manipulate Course Files

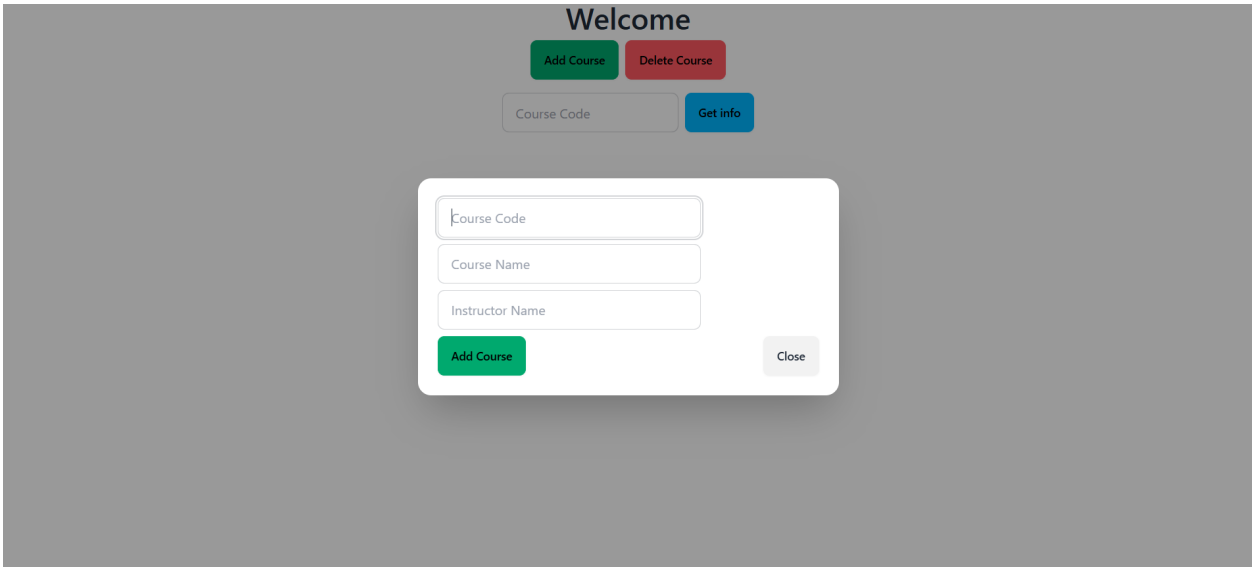

Figure 8: Admin Screen, Add Course Interface

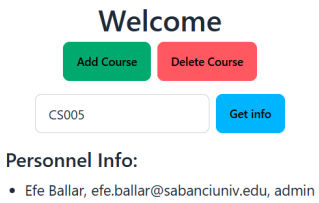

Figure 9: Admin Screen, Course Info Interface

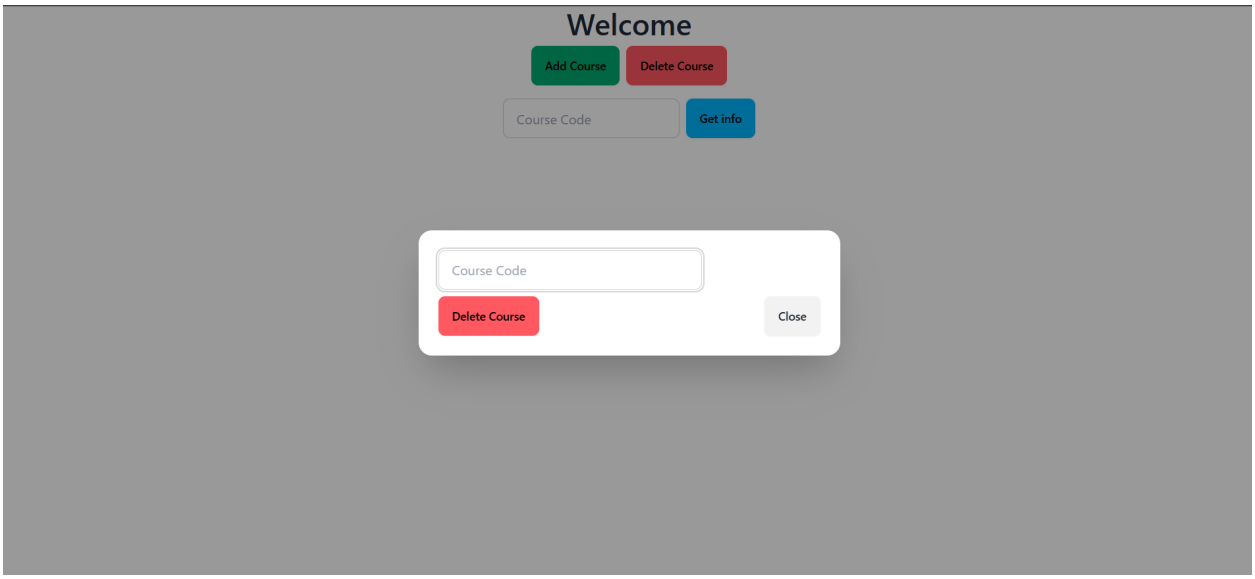

Figure 10: Admin Screen, Delete Course Interface